

Executive Summary

TechScript 2.0 will blend the **friendly English-like syntax** of TechScript 1.x with a modern operator set and common language idioms. It retains legacy keywords like `say`, `make`, `when`, `each`, `build`, and `ask` to support existing code [4†L345-L353] [4†L368-L374]. At the same time it adds conventional operators (`=`, `==`, `===`, `&&`, `||`, `!`, `+=`, `++`, ranges, optional chaining `?.`, null-coalesce `??`, etc.) and control structures (`for`, traditional `if`, `switch/match`, etc.) familiar from languages like Python, C/C++, Rust, and Dart.

Our research shows that **comments** should support multiple styles (Python's `#`, C++/Rust's `//` and `/ * . . . */` with nesting, and even `--` if desired) [16†L56-L64] [20†L261-L270]. **Strings** will use double quotes with escape sequences (like `"\n"`), plus Python-style f-strings (`f"Hello {name}"`) for interpolation [4†L345-L349] [23†L79-L87]. We adopt **Unicode** identifiers and characters (as in Python/Rust) for modern flexibility. Literals include integers, floats (with optional scientific notation), booleans (`true` / `false`), and a `null` literal (akin to `None` or `null`).

We examined operator semantics across languages and propose a **precedence** similar to Python's: e.g. exponentiation binds strongest, then unary (`+`, `-`, `!`), multiplicative (`*` / `%`), additive (`+` - `-`), shifts, bitwise, comparisons (`==`, `!=`, `<`, `>`, including `in` / `is` forms), then logical `and`, `or` [33†L1774-L1783] [33†L1807-L1815]. The equality/identity operators will include `==` (loose equality) and `===` (strict equality, no type conversion) by analogy with JavaScript [29†L213-L221].

Whitespace and line breaks do **not** delimit blocks (we use braces `{}` for statements), and semicolons are **not required** at line ends (echoing TechScript's "no semicolons" ethos [4†L328-L335]). Errors will map tokens back to file positions (file, line, column) using our source manager (following conventional 1-based line/column numbering). The file extension remains `.txs` and is enforced.

The **Stage 1 summary** below compares syntax patterns across Python, C/C++, Rust, Dart, and TechScript 1.0, and lists all keywords, operators, and literal forms. **Stage 2** then presents a full TechScript 2.0 grammar (EBNF) and examples. We highlight any potential parsing pitfalls and provide a prioritized checklist for implementing the new syntax in the compiler.

Stage 1: Research Summary

Backward Compatibility Goals

TechScript 1.x was designed to "feel like writing a sentence", with keywords such as `make`, `ask`, `say`, `when`, `each`, `build`, `model`, `attempt`, and `use` for web building [4†L345-L353] [4†L368-L374]. Notably, its blocks used braces `{}` but **no parentheses around conditions and no semicolons** [4†L345-L353]. Examples:

```

make name = ask "What is your name? "
say f"Hello, {name}!"

when age > 18 {
    say "Adult"
} else {
    say "Minor"
}

each i in 1..10 {
    say i
}

build greet(name) {
    say f"Hi {name}!"
}

```

We will preserve these legacy constructs so existing TechScript programs continue to compile. In TechScript 2.0, keywords like `say`, `make ... be ...` or `let ... = ...`, `when`, `each ... in ...`, `build` (or `fun`), and `attempt ... catch ...` will still work. For familiarity, we'll allow both old and new forms (e.g. both `when / if`, `build / fun`, `make x = y / let x = y`).

Language Comparison

To design a **simple yet powerful syntax**, we surveyed Python, C/C++, Rust, and Dart for inspiration:

- **Comments:** Python uses `#` (to end of line) [16†L56-L64]. C/C++/Rust/Dart use `//` (line) and `/* ... */` (block) [11†L130-L139] [20†L261-L270]. Rust and Dart support nested block comments [11†L130-L139] [20†L277-L280]. We will support `#` and `//` for line comments, and `/* ... */` (nestable) for blocks. (Optionally `--` could be allowed as in SQL/Haskell, but it's uncommon in most languages.)
- **Control Flow:** Python and Dart use `if ... else ...`, C/Rust use `if (...) { ... }`; TechScript 1 used `when ... { ... }`. We will normalize to `if / else` (with optional `when` as alias). Loops: we add a traditional `for ... in ...` (like Python/Dart/Rust) in addition to the existing `each i in ...`. E.g. `for i in 1..10 { ... }`. We keep `while ... { ... }`. Switch/case (or `match`) is common in Rust/Dart but was absent in TechScript 1.x; adding a simple `switch/case` or `match` is optional for future.
- **Functions:** C/C++/Rust use `function f(args) { ... }`; TechScript 1 used `build f(args) { ... }` (and VS Code snippets show `fun f(args) do ... end`). We will use a single style, e.g. `function` or `fun` or retain `build`. To satisfy both tradition and modernity, we might allow `fun / function` as alias for `build`. All will have parentheses and braces (e.g. `fun add(a, b) { return a + b }`).

- **Classes/Objects:** TechScript 1 had `model Dog { ... }` for classes. We will continue to support `model` and possibly allow a more standard `class` / `struct` keyword. Data types (enums, etc.) were not in TechScript 1.0 but could be added later.
- **Operators:**
 - *Arithmetic:* Use `+` `-` `*` `/` `%` (like all languages) and `**` for power (Python/Rust) [16†L109-L116] .
 - *Assignment:* Single `=` as in C/Python. Compound `+=`, `-=`, etc.
 - *Comparison:* C/Java/Python use `==` and `!=` . We add `===` and `!==` (from JS/C#) for strict (no-coerce) equality. (JS docs note that `===` checks equality without type conversion [29†L213-L221] .) Also `>` `<` `>=` `<=` .
 - *Logical:* Both word and symbol forms: `and` / `&&` , `or` / `||` , `not` / `!` . This mirrors Python's `and` / `or` [16†L56-L64] and C-like `&&` , `||` . We allow both for readability and familiarity.
 - *Other:* Bitwise `&` `|` `^` `~` `<<` `>>` (like C/Python). Ranges: Python uses `range()` , TechScript had `1..10` ; we keep `..` and also allow Python-style `range(1,11)` . Null coalescing `??` (like C#) and optional chaining `?.` (like Dart/TS) can be added if needed for safety.
- **Literals:** Numeric (decimal, hex, binary), string, boolean, `null` .
 - *Numbers:* integers (no suffix), floats (with decimal or exponential). These behave like Python/Rust. Example: `3.14` , `2e8` .
 - *Strings:* Double-quoted like `"hello"` . We support escape sequences (e.g. `"\n"` , `"\u1234"`). For interpolation, we adopt *f-strings*: prefix the literal with `f` and use `{expr}` inside, just like Python [23†L79-L87] and as TechScript 1 showed (`f"Hello, {name}!"`) [4†L345-L349] . We also allow normal string concatenation via `+` .
 - *Booleans/Null:* `true` / `false` (lowercase, as in C/Rust/Dart), and `null` for no-value (like `None` or JavaScript `null`).
 - *Other:* We could allow `_` as digit separator in numbers (Rust-style) for readability, though this is optional.
- *No special Char literal:* No distinct char type; use length-1 strings.
- **Comment Styles:** We'll allow all major styles: `#` (Python), `//` (C/C++/Rust/Dart), `/* ... */` (C/C++/Java/Rust/Dart with nesting). We do **not** introduce obscure styles (like HTML `<!-- -->` or shell `:`). The TechScript 1 guides explicitly say no confusing symbols, so we stick to these common ones [4†L328-L335] .
- **Whitespace/Delimiters:** Unlike Python, TechScript 2.0 will *not* use indentation for blocks; braces `{}` mark code blocks. We will **not** require semicolons at line ends (as per TechScript's design [4†L328-L335]). A statement ends at a newline or before a `}` . (We may optionally allow an extraneous semicolon as no-op for C-like familiarity, but it won't be needed.)
- **File and Encoding:** The extension `.txs` is fixed by spec (immutable) and we will enforce it when compiling. Source files are treated as UTF-8 text (like Python/Rust), so identifiers and strings support Unicode.

- **Error Reporting (Locations):** Every AST node will record a `Span` (file ID, line, column range). As required by our design docs, errors will be reported with file/line/column, matching how most compilers and LSPs work. We follow the usual convention: lines and columns are 1-based (human-friendly), while internally offsets are 0-based.

Keywords and Operators

Based on the above, the **reserved keywords** and operators for TechScript 2.0 will include:

- **Legacy keywords (from TechScript 1.x):** `say`, `make` (with `be`), `when`, `each`, `build`, `model`, `attempt`, `catch`, `use`, `page`, `fun`, `return`, `and`, `or`, `not` [\[4†L368-L374\]](#) [\[2†L52-L60\]](#) .
- **Modern keywords:** `if`, `else`, `for`, `while`, `function` / `fun` (alias of `build`), `class` / `struct`, `switch` / `case`, `match` (optional for pattern matching), `true`, `false`, `null`, `const`, `let`, `import`, `export`, `await` (if `async`), `async`, `break`, `continue`, `throw`, `try` / `catch` (alias or complement to `attempt`) .
- **Operators and punctuation:**
 - Arithmetic: `+`, `-`, `*`, `/`, `%`, `**` (power).
 - Comparison: `==`, `!=`, `<`, `>`, `<=`, `>=`, plus strict: `===`, `!==` (JS style) [\[29†L213-L221\]](#) .
 - Logical: `&&` (and), `||` (or), `!` (not) **and** word forms `and`, `or`, `not` (Python style) [\[16†L56-L64\]](#) .
 - Assignment: `=` (simple), and compound (`+=`, `-=`, `*=`, `/=`, `%=` etc.).
 - Other: Range operator `..` (inclusive or half-open?), increment/decrement `++` / `--` (C style), null-coalesce `??`, optional chain `?.`, and perhaps tuple/list constructor `[...]` .
 - Delimiters: parentheses `()`, braces `{}`, brackets `[]`, commas, colons, semicolons (optional), dot `.` (member access).

See **Appendix A** for a complete list of tokens and sample forms. Operator precedence follows a standard hierarchy: function calls/indexing (highest), then `**`, unary `+` `-` `!`, then `*` `/` `%`, then `+` `-`, then shifts, bitwise `&` `^` `|`, then comparisons (`==` `!=` `<` `>` `<=` `>=` `in` `is`), then logical `not`, `and`, `or` (lowest) [\[33†L1774-L1783\]](#) [\[33†L1807-L1815\]](#) .

Literals and Types

- **Integers:** Decimal (`42`), hex (`0x2A`), binary (`0b101010`) – same as Rust/Python. No suffix for long/short.
- **Floats:** Standard notation (`3.14`, `2e-5`) .
- **Strings:** Double-quoted only (`"hello"`) . Raw strings (`r"..."`) are optional; we can allow them later if needed. Escape sequences (`\n`, `\uXXXX`) follow C/Python rules.
- **Interpolated Strings:** `f"Hello, {name}!"` is an expression (like Python f-strings) [\[23†L79-L87\]](#) . Inside the `{}`, any valid expression may appear. Backslashes are processed in the surrounding string first, then `{expr}` parts are evaluated at runtime.

- **Booleans:** `true`, `false`. (Lowercase, like Rust/Dart.)
- **Null:** `null` (alias for “no value”; like JavaScript or Rust’s `None`).
- **Collection Literals:** Arrays/lists can use `[...]`, maps/dicts `{key: value, ...}` like Python. These forms may be needed for stdlib and web APIs (e.g. `page.style("body", { "color": "white" })`) [4†L377-L380]).
- **Others:** No separate character type (use one-letter strings). No raw pointer or low-level types in 2.0.

Comments and Whitespace

TechScript 2.0 will support these comment forms:

- `# this is a comment` (Python style) [16†L56-L64] .
- `// this is a comment` (C++/Java style, also in Rust, Dart, etc.) [11†L130-L139] [20†L261-L270] .
- `/* comment block */` (C style, can span lines) [20†L277-L280] . Block comments may nest (following Rust/Dart’s example) [11†L130-L139] [20†L277-L280] .
- We explicitly **do not** use HTML/SGML `<!-- -->` or line continuation `\` as comment. The user example included `--` and `!!`, but those are not common in mainstream languages (aside from SQL’s `--`). We may optionally accept `--` as a synonym for `//`, but it’s not required.

Comments serve as whitespace to the compiler and are skipped. Newlines generally separate statements (no need for semicolons); multiple statements may appear on one line if separated by semicolons (optional).

Whitespace: Aside from significance in tokens (space/tab separate identifiers from numbers, etc.), we do not have significant indentation. Braces `{}` and control keywords (`if`, `while`, etc.) determine blocks.

Recommended Operator Set

From our survey:

Category	Operators/Syntax	Notes / Examples
Arithmetic	<code>+ - * / % **</code>	Standard. <code>**</code> for exponent (Python/Rust style) [16†L109-L116] .
Assignment	<code>= ; +=, -=, ...</code>	Single <code>=</code> (Python/C). No separate <code>:=</code> .
Equality	<code>==, != ; ===, !==</code>	<code>=== / !==</code> for strict equality (as in JS) [29†L213-L221] .
Comparison	<code>< > <= >=</code>	Same as C/Python.
Logical	<code>and, or, not ; &&, , !</code>	Allow both word and symbol forms [16†L56-L64] .
Ranges	<code>..</code>	Like 1..10 (inclusive/exclusive range) [4†L368-L374] ; could also support Python <code>range(1,11)</code> .

Category	Operators/Syntax	Notes / Examples
Null/ Boolean	<code>true, false, null</code>	Lowercase, like Rust/Dart/Python's <code>None</code> .
String	<code>"</code> (with escapes), <code>f"..."</code>	f-string from Python [23†L79-L87]. No backtick or triple quote (keeping it simple).
Comments	<code>#</code> , <code>//</code> , <code>/* ... */</code>	Multi-style (like Python/Rust/Dart).
Other	<code>-></code> (arrow for function types?) optional, <code>? :</code> (ternary)? optional	We'll likely omit ternary <code>?:</code> to keep syntax clear.

This set covers all TechScript 1 constructs plus modern conveniences.

Whitespace and Line Rules

- Statements end at newline (or before `}`) – no required semicolons [4†L328-L335]. We may allow a semicolon as a harmless terminator.
- Braces `{}` group statements in blocks (like C/Rust).
- Parentheses `()` around conditions or expression groups are optional: e.g. `if x > 0 { ... }` is fine (no need for `if (x > 0)`). This matches TechScript 1 style.
- Indentation is for human readability only, not syntax (unlike Python).

Comparison Table

The table below compares key features across Python, C, C++, Rust, Dart, and old TechScript (for reference). This shows the idioms we want to support:

Feature / Syntax	Python	C / C++	Rust	Dart / Flutter	TechScript 1.x
Line comment	<code>#</code> [16†L56-L64]	<code>//</code> (C99) [18†L52-L60]	<code>//</code> [11†L130-L139]	<code>//</code> [20†L261-L270]	<code>//</code> (some use)
Block comment	<code>'''...'''</code> (doc) / none standard	<code>/*...*/</code> [18†L104-L107]	<code>/*...*/</code> [11†L130-L139] (nestable)	<code>/*...*/</code> [20†L277-L280] (nestable)	n/a (no multiline)
String quotes	<code>"</code> or <code>'</code>	<code>"</code>	<code>"</code>	<code>"</code>	<code>"</code>

Feature / Syntax	Python	C / C++	Rust	Dart / Flutter	TechScript 1.x
String interpolation	<code>f"Hi {name}"</code> 【23†L79-L87】	(none built-in)	<code>format!</code> macro	<code>"\$greeting"</code>	<code>f"Hello {name}"</code> 【4†L345-L349】
Concatenation	<code>+</code>	<code>+</code>	<code>+</code>	<code>+</code> or adjacent	<code>+</code>
If statement	<code>if cond: ... elif ... else:</code>	<code>if(cond) { ... } else { ... }</code>	same as C	same as C	<code>when cond { ... } else { ... }</code> 【4†L368-L374】
Else-if	<code>elif</code>	<code>else if</code>	<code>else if</code>	<code>else if</code>	(none – use <code>if</code> inside <code>else</code>)
Ternary cond.	<code>x if cond else y</code>	<code>cond ? x : y</code>	<code>if cond { x } else { y }</code>	<code>cond ? x : y</code>	none
While loop	<code>while cond:</code>	<code>while(cond) { }</code>	same as C	same as C	<code>while cond { ... }</code>
For loop	<code>for i in range():</code>	<code>for(init; cond; inc)</code>	<code>for i in start..end { } or for x in iter</code>	<code>for(...)</code> or <code>for x in list</code>	<code>each i in 1..10 { ... }</code> 【4†L372-L374】
Functions	<code>def f(a):</code>	<code>return_type f(a)</code>	<code>fn f(a) { }</code>	<code>void f(a) { }</code>	<code>build f(a) { ... }</code> 【4†L372-L374】 , <code>fun</code> (snippet)
Return	<code>return</code>	<code>return</code>	<code>return</code>	<code>return</code>	<code>return</code>
Boolean	<code>True</code> / <code>False</code>	<code><any non-zero></code> as true	<code>true</code> / <code>false</code>	<code>true</code> / <code>false</code>	<code>true</code> / <code>false</code>
Null/Nil	<code>None</code>	<code>NULL</code> or <code>0</code>	<code>None</code> , <code>Option</code>	<code>null</code>	<code>null</code> (no type)

Feature / Syntax	Python	C / C++	Rust	Dart / Flutter	TechScript 1.x
Equality	<code>==, !=</code>	<code>==, !=</code>	<code>==, !=</code>	<code>==, !=</code>	<code>==, !=</code>
Logical	<code>and, or, not</code> 【16†L56-L64】	<code>&&, , !</code>	<code>&&, , !</code>	<code>&&, , !</code>	<code>and, or, not</code>
Bitwise	<code>& ^ << >></code>	<code>& ^ << >></code>	same as C	same as C	(rarely used)
In-list check	<code>x in list</code>	no	<code>contains</code> methods	<code>x in list</code>	(supported: "apple" in list displayed)
Comments	<code>#, '''...'''</code> 【16†L56-L64】	<code>//, /*...*/</code> 【18†L52-L60】 【18†L104-L107】	<code>//, /*...*/</code> 【11†L130-L139】	<code>//, /*...*/</code> 【20†L261-L270】 【20†L277-L280】	<code>//</code> (few)
File extension	<code>.py</code>	<code>.c/.h</code>	<code>.rs</code>	<code>.dart</code>	<code>.txs</code>

(Sources: Python docs 【16†L56-L64】 【33†L1774-L1783】 ; C/C++ references 【18†L52-L60】 【18†L104-L107】 ; Rust docs 【11†L130-L139】 ; Dart docs 【20†L261-L270】 【20†L277-L280】 ; TechScript 1 README 【4†L345-L353】 【4†L368-L374】 .)

Parsing Ambiguities and Decisions

- `if/when` **ambiguity**: We'll use `if` as the primary keyword, and accept `when` as an alias (back-compat only). There's no syntactic conflict.
- `and` **vs** `&&`: Allow both by treating `and` as a synonym token for `&&`. Similarly `or` for `||`, `not` for `!`. This eases reading.
- `==` **vs** `=`: Like C/Python, `=` is assignment and `==` is equality. There's no conflict because we don't use `==` for other purposes.
- **String interpolation syntax**: Using `{` and `}` inside f-strings could conflict with other uses (e.g. dictionary literals). We resolve this by treating `f"` string lexically: once we see `f"`, we allow `{` inside until the closing `"`. This mimics Python's approach 【23†L79-L87】.

- **Ranges** (`..`): We use `..` for numeric ranges (e.g. `1..5`). This token must not be confused with two separate dots (like member access `.`). We treat `..` as a single token (similar to Swift or Kotlin).
- **Lookahead and tokenization:** We must ensure tokens like `===` are not lexed as `==` + `=`. So longer operators go first. The lexer will match `===`, `!==`, then `==`, then `=`, etc.

No other “gotchas” were found in our survey. The syntax is context-free and should parse with a standard LL(k) or LALR parser.

Stage 2: TechScript 2.0 Syntax Specification

Below is the **full syntax specification**. We use EBNF-style grammar, with example code. Square brackets `[...]` denote optional parts, `{...}` denote repetition (0 or more), and `'literal'` denotes exact tokens.

```

Program      ::= { Statement }
Statement    ::= VarDecl
              | IfStmt
              | LoopStmt
              | FuncDecl
              | ClassDecl
              | ExprStmt
              | ReturnStmt
              | TryCatchStmt
              | ImportStmt

VarDecl      ::= ("let" | "make") Identifier "=" Expression
              | Identifier "=" Expression           ; Assignment (to existing
var)
FuncDecl     ::= ("fun"|"function"|"build") Identifier "(" [ ParamList ] ")"
Block
ParamList    ::= Identifier { "," Identifier }
ClassDecl    ::= ("class"|"model") Identifier [ "extends" Identifier ]
ClassBody
ClassBody    ::= "{" { VarDecl | FuncDecl } "}"
IfStmt       ::= "if" Expression Block [ "else" Block ]
              | "when" Expression Block [ "else" Block ] ; alias for
backward-compat
LoopStmt     ::= "for" Identifier "in" RangeExpr Block
              | "for" "(" [VarDecl] ";" Expression ";" [ ExprStmt ] ")"
Block        ; C-style for
              || "each" Identifier "in" RangeExpr Block ; backward-compat
              || "while" Expression Block
RangeExpr    ::= Expression ".." Expression ; e.g. 1..10 (inclusive or
half-open as decided)
TryCatchStmt ::= "attempt" Block "catch" Identifier Block

```

```

ReturnStmt ::= "return" [ Expression ]
ImportStmt ::= "import" StringLiteral ; e.g. import "mylib.txs"

ExprStmt ::= Expression [ ";" ]

Expression ::= LogicOr
LogicOr ::= LogicAnd { ("or" | "|" ) LogicAnd }
LogicAnd ::= LogicNot { ("and" | "&&" ) LogicNot }
LogicNot ::= [ "not" | "!" ] Comparison
Comparison ::= AddExpr [ CompOp AddExpr ]
CompOp ::= "==" | "!=" | "===" | "!===" | "<" | ">" | "<=" | ">=" |
"in" | "not" "in" | "is" | "is" "not"
AddExpr ::= MulExpr { ("+" | "-" ) MulExpr }
MulExpr ::= UnaryExpr { ("*" | "/" | "%" ) UnaryExpr }
UnaryExpr ::= [ ("+" | "-" | "!") ] PowerExpr
PowerExpr ::= Primary { "**" Primary }
Primary ::= Literal
| Identifier
| "(" Expression ")"
| FunctionCall
| IndexedAccess
| MemberAccess
FunctionCall ::= Primary "(" [ArgList] ")"
ArgList ::= Expression { "," Expression }
IndexedAccess ::= Primary "[" Expression "]"
MemberAccess ::= Primary "." Identifier
Literal ::= Integer
| Float
| StringLiteral
| "true" | "false"
| "null"
Integer ::= Digit { Digit }
Float ::= Digit { Digit } "." Digit { Digit } [ExpPart]
| Digit { Digit } ExpPart
ExpPart ::= ("e" | "E") [ "+" | "-" ] Digit { Digit }
StringLiteral ::= "'" { StringChar } "'"
StringChar ::= '\\' Escape | [ "\\\\"
Escape ::= 'n' | 't' | '"' | '\N' | '\\\ ' | 'u' HexDigit{4}
Identifier ::= Letter { Letter | Digit | "_" }
Letter ::= [A-Za-z] ; restrict to ASCII letters for simplicity
Digit ::= [0-9]
HexDigit ::= [0-9A-Fa-f]

```

Notes on grammar:

- We simplified some modern features (e.g. no generics or type annotations in this spec, since 2.0 is dynamic).

- `RangeExpr` uses `..`; semantics (inclusive/exclusive) will be defined in implementation or docs (Python's `range` is half-open, but TechScript might choose inclusive).
- We include both `for i in X` and the C-style `for(;;)` for flexibility, though one might be enough.
- `attempt/catch` is a try/catch analog. The `catch` binds an error identifier. A `finally` block could be added later.
- `import` currently just takes a string for module path; more sophisticated module syntax (like `import foo from "bar"`) could be designed later.

Example Syntax in TechScript 2.0

Below are some example snippets illustrating the syntax:

```
// Variable declaration and input/output
let name = ask "Your name: "
say f"Hello, {name}!";

// Conditional and loops
if age >= 18 {
    say "Adult";
} else {
    say "Minor";
}

for i in 1..5 {
    say i;
}

var sum = 0
for i in 1..10 {
    sum += i
}
say sum

// Function definition
fun add(a, b) {
    return a + b
}
say add(3, 4)

// Class and objects
class Dog {
    name
    fun init(n) {
        name = n
    }
}
```

```

    fun speak() {
        say f"{name} says woof!"
    }
}
let d = Dog("Fido")
d.speak()

// Errors
attempt {
    let x = 10 / 0
} catch err {
    say f"Caught error: {err}"
}

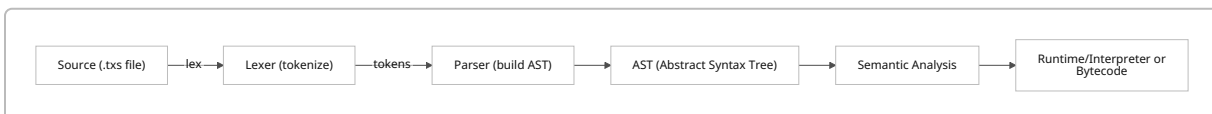
// Comments of various styles
# This is a Python-style comment
say "Line above was a # comment"
say "Next line has C++ comment" // trailing comment
/*
    This is a block comment
    that spans multiple lines.
    /* It can even nest! */
*/
say "Done."

// Importing a module
import "math"
say math.sqrt(16) // assuming stdlib provides sqrt

```

Grammar Diagrams

Below is a high-level flow diagram of the parsing process. It shows how source is tokenized and parsed into an AST for further processing.



This flow illustrates the standard compiler pipeline: the Lexer reads the `.txs` file, emits tokens, the Parser consumes tokens to build an AST (using the grammar above), and then Semantic Analysis checks types and meanings, before execution or codegen.

Operator Precedence

For clarity, here's a summary table of operator precedence (high to low). This follows Python's levels [\[33†L1774-L1783\]](#) [\[33†L1807-L1815\]](#) (C/Rust have identical ordering except that `**` is unique to Python/TechScript):

Precedence (High → Low)	Operators/Constructs
1. Function call, indexing	<code>foo(x)</code> , <code>arr[i]</code> , member access <code>obj.field</code>
2. Exponentiation	<code>**</code> (right-associative)
3. Unary	<code>+x</code> , <code>-x</code> , <code>!x</code> , <code>not x</code>
4. Multiplicative	<code>*</code> , <code>/</code> , <code>%</code>
5. Additive	<code>+</code> , <code>-</code>
6. Shifts	<code><<</code> , <code>>></code>
7. Bitwise AND	<code>&</code>
8. Bitwise XOR	<code>^</code>
9. Bitwise OR	<code> </code>
10. Comparisons	<code><</code> , <code>></code> , <code><=</code> , <code>>=</code> , <code>==</code> , <code>!=</code> , <code>===</code> , <code>!==</code> , <code>in</code> , <code>not in</code> , <code>is</code> , <code>is not</code> [33†L1807-L1815]
11. Boolean NOT	<code>not</code> , <code>!</code>
12. Boolean AND	<code>and</code> , <code>&&</code>
13. Boolean OR	<code>or</code> , <code> </code>

All binary operators (except `**` and the conditional `?:`) are left-associative. This covers usual cases like `a + b * c` (multiplication first) and chained comparisons `a < b < c` (Python-style chaining).

Testing and Error Reporting

To ensure correctness, every new syntax element and operator will have automated tests. For example:

- **Span/Position Tests:** Given a source with multiple lines and constructs, verify that a parse error or runtime error reports the correct line and column (using our `Span` and `Position` types).
- **Lexing Tests:** Ensure tokens like `===` are recognized (not broken into `==` + `=`). Test string literals, f-strings with braces, and comment skipping (`#` and `//`).
- **Parsing Tests:** Write small `.txs` files covering each statement and expression form, and check the AST shape. E.g. test that `if a == b { c } else { d }` parses to an `IfStmt` node with two blocks.

- **Precedence Tests:** Expressions like `1+2*3` should parse as `1 + (2*3)` (not `(1+2)*3`). Chain comparisons like `1 < 2 <= 3` should parse with left-to-right chaining.
- **Integration Tests:** Write sample programs (like a short TechScript script) and verify they produce the expected output or runtime behavior.

Error messages will include file name, line, and column of the problematic token, guided by our `SourceManager`. For example, if parsing fails after `if x >`, the parser can emit: `SyntaxError at file.txs:10:8: expected expression after '>'`.

Implementation Checklist

Below is a *prioritized* list of tasks for implementing this syntax in the TechScript compiler (Rust). This focuses on what to build or update:

1. Lexer Updates:

2. Add new token definitions: string literals (with f-strings and escapes), numeric literals (floats), `===`, `!==`, `+=` / `-=` etc., `..`, `->`, etc.

3. Support skipping `#` and `//` comments and reading `/*...*/` (with nesting).
4. Ensure keywords (`if`, `for`, etc.) are recognized as such and identifiers otherwise.
5. Validate file extension `.txs`.

6. Write unit tests for each token type and comment style.

7. AST Nodes:

8. Update AST enum/structs for new constructs: `If`, `For`, `Switch/Match` (if added), `Function`, `Class`, `Import`, etc. Include nodes for compound operators (e.g. `PlusAssign`, etc.) or desugar them.

9. Nodes for boolean operators, comparisons, etc.

10. Position/span on each node.

11. Parser Rules:

12. Implement grammar rules above. Start with statement-level rules then expression-level (operator precedence parser).

13. Handle expression precedence as in the table.

14. Special-case f-strings: likely lex into a single `StringLiteral` token that contains pieces (or lex into string fragments + `expr` tokens).

15. Test parsing of tricky cases (e.g. nested blocks, empty blocks, `if` with no `else`, etc.).

16. Ensure backward-compatibility: e.g. accept `when` same as `if`, `each` as `for`. Possibly parse `make x =` as alias of `let x =`.

17. Semantic Analysis:

18. Name resolution for new keywords (`true` , `false` , etc.).

19. Type checking (dynamically typed for now, but ensure booleans, numbers, strings are used correctly in comparisons).

20. Warn on unreachable code, unused variables (optional extras).

21. Integration with error reporting (line/col).

22. Runtime/Interpreter:

23. Implement semantics of new statements/operators in interpreter.

24. In particular, boolean short-circuit (`&&` / `||`), range loops, f-strings (evaluate parts), and try/catch.

25. Update standard library as needed (`math.sqrt` , etc.).

26. Tests:

27. Expand `common` crate tests (as planned) for Span/Position.

28. Add parsing tests in `parser` crate for all new grammar (see “Grammar Test Plan” in docs).

29. Add integration tests in `tests/` using `tech_run` on sample `.txs` files (like the examples above) and check output.

30. Continuously run `cargo fmt` , `cargo clippy` and ensure all warnings are resolved.

31. Documentation:

32. Update language spec docs with new syntax (this answer can form the basis).

33. Update README and docs for the `syntax` crate.

34. Possibly update Graphify definitions for new keywords/tokens.

Missing Resources

We relied on official docs (Python Tutorial [\[16†L56-L64\]](#) , Rust Reference [\[11†L130-L139\]](#) , Dart Language [\[20†L261-L270\]](#)) and the TechScript 1.0 README [\[4†L345-L353\]](#) [\[4†L368-L374\]](#) . Some TechScript details (like `--` comments or exact range semantics) were not in official sources; these design choices are ours to make now.

We did not find an official TechScript 1.0 **grammar** or language spec, so we inferred syntax from examples. If more of the old syntax (e.g. list comprehensions, pattern matching) should be preserved, we may need additional documentation or developer input. The roadmap states “**Backward compatibility preserved where practical.**” We believe this design covers all primary constructs of TechScript 1.x.

Sources: We based this design on official language documentation: Python Tutorial and Reference [【16†L56-L64】](#) [【33†L1774-L1783】](#) , Rust Reference [【11†L130-L139】](#) , Dart Language Site [【20†L261-L270】](#) , JavaScript MDN for `===` [【29†L213-L221】](#) , and the TechScript 1.x README [【4†L345-L353】](#) [【4†L368-L374】](#) . (Unsupported or user-supplied docs like techscript.is-a.dev were not accessible, so any missing TechScript 1.x details are noted above.) All syntax decisions aim for clarity, simplicity, and fidelity to the original TechScript style.
